



Tecnológico de Monterrey

Inteligencia artificial avanzada para la ciencia de datos

Reporte

Implementación de un modelo de regresión lineal
mediante Descenso de Gradiente

Mauricio Olguín Sánchez
A01711522

Agosto 2026

Índice general

Capítulo 1

Realizando la implementación «a manita»

1.1. Sobre la base de datos seleccionada

Después de haber investigado en páginas como [UC Irvine](#), [Kaggle](#), el [World Bank Open Data](#), entre otras bases de datos, en la página de [OpenML](#) encontré un *dataset* que se me hizo interesante. Tras aplicar varios filtros, encontré la base de datos [Airlines_DepDelay_10M](#).

Esta base de datos está construida a partir de datos provenientes de la Oficina de Estadísticas de Transporte de los Estados Unidos (*Bureau of Transportation Statistics*, BTS), específicamente, de la base de datos *Airline On-Time Performance Data*. La base de datos seleccionada contiene 10,000,000 de registros y 10 variables, y corresponde a un problema de regresión. A continuación, presento una breve descripción general del *dataset*:

Propósito: Predecir el tiempo de retraso en la salida de los vuelos, medido en minutos (*Departure Delay*).

Fecha de los datos: Desde 1987 hasta 2013.

Formato: ARFF (*Attribute-Relation File Format*).

Tipo de tarea: Regresión.

Número de registros: 10,000,000.

Número de variables: 10.

Aunque OpenML proporciona información sobre cada variable, dentro de la fuente original (en el *Bureau of Transportation Statistics*), existe una descripción más detallada sobre los datos:

DepDelay - numerico. Diferencia, en minutos, entre la hora de salida programada y la hora de salida real del vuelo. Los valores negativos indican que el vuelo salió antes de lo programado.

Month - numerico. Mes en el que se realizó el vuelo, con valores del 1 al 12, donde 1 corresponde a enero y 12 a diciembre.

DayofMonth - numerico. Día del mes en el que se realizó el vuelo, con valores del 1 al 31.

DayOfWeek - numerico. Día de la semana en el que se realizó el vuelo, representado mediante valores del 1 al 7.

CRSDepTime - numerico. Hora de salida programada del vuelo según el sistema de reservas computarizado de la aerolínea (CRS), expresada en hora local con formato HHMM.

CRSArrTime - numerico. Hora de llegada programada del vuelo según el sistema de reservas computarizado de la aerolínea (CRS), expresada en hora local con formato HHMM.

UniqueCarrier - categorico. Código único utilizado para identificar a la aerolínea que opera el vuelo.

Origin - categorico. Código del aeropuerto de origen del vuelo.

Dest - categorico. Código del aeropuerto de destino del vuelo.

Distance - numerico. Distancia total del vuelo, expresada en millas.

que crezca la dimensionalidad del conjunto de datos.

En contraste, las variables *UniqueCarrier*, *Origin* y *Dest* son explícitamente categóricas, pero serán excluidas del modelo. Su transformación (utilizando la misma técnica de antes) produciría un incremento considerablemente mayor en la cantidad de variables. Principalmente, tomo esta decisión con el objetivo de mantener un tiempo de ejecución razonable y centrar el proyecto en la implementación manual del algoritmo de regresión.

1.2. Proceso de ETL

1.2.1. *Extraction*

Como ya lo he mencionado antes, los datos se han obtenido del siguiente link: <https://www.openml.org/search?type=data&status=active&id=42728>

Sin embargo, si se quisiera obtener más datos actuales, habría que descargarlos (sea de forma manual o creando algún *script*) desde el siguiente link: <https://transtats.bts.gov/PREZIP/>.

Aquí, se recopilan todos los datos desde 1987 hasta junio de 2026.

1.2.2. *Transforming*

1.2.3. Sobre los atributos *Month*, *DayofMonth* y *DayOfWeek*

Las variables *Month*, *DayofMonth* y *DayOfWeek*, aunque se encuentran representadas numéricamente en el conjunto de datos, las trataré como variables categóricas. Esto se debe a que, en realidad, los valores numéricos representan categorías y no implican necesariamente una relación lineal entre ellas. Por ejemplo, el mes representado por el valor 12 no debe interpretarse como una categoría doce veces mayor que el mes representado por el valor 1.

Para representar estas variables dentro del modelo de regresión, utilizaré la técnica de *One-Hot Encoding*. Aunque esto implicará

Sobre los atributos *CRSDepTime* y *CRSArrTime*

Las variables *CRSDepTime* y *CRSArrTime* representan las horas programadas de salida y llegada de los vuelos. En el conjunto de datos original, estas se presentan en formato numérico continuo basado en la estructura HHMM (por ejemplo, las 13:20 se representan como 1320).

Sin embargo, esta representación no es adecuada para ser procesada de forma directa por un modelo de regresión lineal. El formato HHMM introduce discontinuidades matemáticas artificiales debido a los cambios de hora. Por ejemplo, los valores 1059 (10:59) y 1100 (11:00) tienen una separación temporal real de apenas 1 minuto; no obstante, si el modelo consume los datos en su formato original, interpretará una distancia matemática de 41 unidades. Esta magnitud falsa distorsiona los gradientes durante el entrenamiento, asumiendo una separación temporal mucho mayor a la real.

Para corregir este desajuste y linealizar la variable, se transformaron ambos atributos utilizando como referencia la medianoche absoluta. Para ello, se separaron las horas y los minutos del valor original y se aplicó la siguiente ecuación:

$$\text{Minutos}_{\text{desde la media noche}} = 60 \times \text{Hora} + \text{Minutos}$$

Esta transformación garantiza que, la distancia numérica entre cualquier par de ho-

ras, refleja entonces con exactitud la distancia real. 1059 se convierte en 659 y 1100 en 660, logrando la diferencia exacta de 1 unidad. Además, se mantiene la jerarquía de los horarios a lo largo del día en un espectro entre 0 a 1439.

Sobre el formato del archivo de los datos

Al empezar a realizar una exploración de los datos, cada vez que tenía que ejecutar el programa para realizar algo, tardaba aproximadamente 45 segundos en sólo cargar los datos. Investigando opciones para solucionar este problema, me encontré con el formato `parquet`. Este formato está precisamente optimizado para mejorar el manejo de datos complejos, el rendimiento de las consultas que se hacen y la eficiencia del almacenamiento [1].

Al informarme que esta integración me resultaría óptima para poder realizar más pruebas más rápidas, la he implementado. Así pues, el archivo ha pasado de tener un peso de 385 MB a sólo 56 MB; además de que el tiempo de carga se redujo drásticamente de 45 segundos a 2 segundos o menos en la carga de datos.

1.2.4. Loading

Una vez hechas las transformaciones anteriores, ahora dividí el tamaño de mi *dataset* en tres principales secciones: *train*, *validation* y *test*. Al ser la cantidad de datos tan alta, podría realizar varias particiones para *validation*, sin embargo, en esta primera parte en donde me enfoco aprender el algoritmo, sólo haré una vez esta división. Así pues, utilizaré una proporción de 98 % de los datos para *train*, 1 % para *validation* y 1 % para *test*. [2]

1.3. Exploratory Data Analysis

Hice un primer acercamiento general a los datos simplemente realizando un gráfico de pares (*pairplot*), el cuál resultó en el siguiente: (ver figura ??)

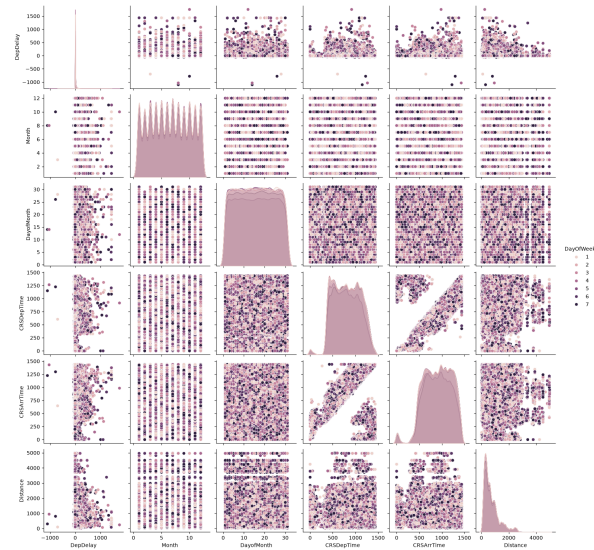
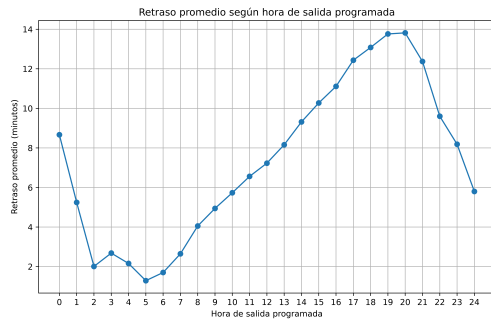


Figura 1.1: *Pair plot* para visualizar la mayoría de los datos comparados entre sí. No se graficaron los 10 millones de puntos por el tiempo de computo requerido, sólo se graficaron un millón. Se añadió también que, para cada punto, se etiquetara por su día de la semana (*DayOfWeek*) para intentar de visualizar alguna zona de interés así como la distribución de los mismos.

De estos gráficos, se puede observar que, cuando se comparan los datos entre *DepDelay*, se pueden apreciar una posible zona donde la mayoría de los datos recaen, sin embargo, también es interesante ver que existen outliers negativos, es decir, que hubo días en donde un vuelo se adelantó a la hora de su salida. También, podemos observar que entre las variables de *CRSDepTime* y *CRSArrTime* existe una clara agrupación de los datos, por lo que valdría la pena volver a realizar este gráfico pero ahora también comparándolo con la distancia final y/o con el tiempo de retraso.

Ahora que tenemos el panorama general de los datos, podría pasar a realizar gráficas específicas que me cuenten un poco más sobre los mismos datos. Al contar con los datos de

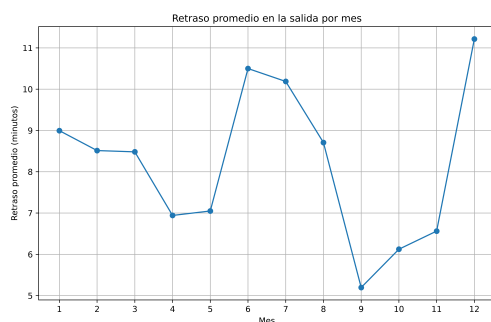
los días de la semana, días del mes, meses del año y las horas en las que se retrasan, decidí hacer un gráfico con los minutos promedio por cada serie de tiempo para ver cómo es que evolucionan los retrasos durante cada periodo de tiempo (ver figura ??)



(a) Promedios de retrasos por cada hora.



(b) Promedios de retrasos por cada día de la semana.



(c) Promedios de retrasos por cada mes.

Figura 1.2: Promedios de retrasos por periodos de tiempos.

De cada uno, puedo destacar lo siguiente:

- Para los retrasos por cada hora del día se puede apreciar que, si empezamos a ver la gráfica desde las 5 de la mañana (el punto más bajo en promedio de retrasos), por cada hora que pasa va incrementando el retraso en cada vuelo

hasta llegar al punto máximo hasta las 7 de la tarde (19 horas).

- Algo similar a lo anterior pasa a lo largo de la semana pues, a medida que se acerca el fin de semana, los vuelos cada día se retrasan más, hasta que el viernes llega a su punto máximo.
- Por último, durante el verano e inicios de años es cuando existen más retrasos en los vuelos. Aunque esta información me hace pensar que coincide con periodos vacacionales, no puedo afirmar con la información que tengo que esta sea la razón.

Sin embargo, es importante destacar que estos valores podrían estar altamente afectados por los *outliers* que existen para cada serie de tiempo. Para confirmar esto, vale entonces la pena realizar el histograma específico con los retrasos (ver figura ??).

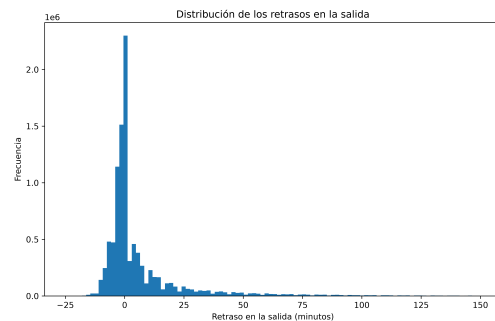


Figura 1.3: Histograma con retrasos en los vuelos. **NOTA:** La escala de este histograma se ha recortado de -25 a 150. Esto debido a que los *outliers* (tanto positivos como negativos) son tan grandes y muy pocos que reducen mucho la visibilidad de la gran mayoría de los datos.

Es interesante observar que la inmensa mayoría de vuelos está a tiempo, ya que es en donde más se concentran los datos. Sin embargo, también es interesante ver que existen vuelos que se adelantan.

Por último, el gráfico más interesante en donde podemos comparar y darnos cuenta de la relación lineal entre las variables es una matriz de correlación (ver figura ??)

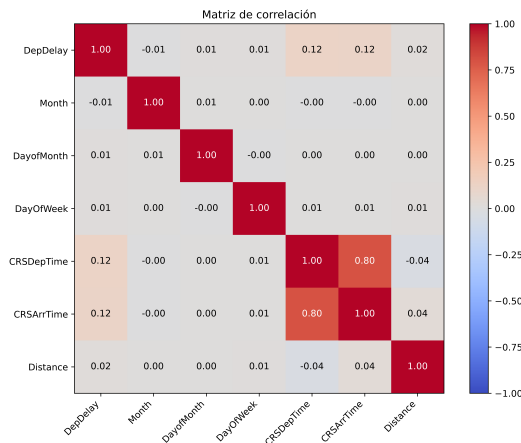


Figura 1.4: Matriz de correlación.

Como se puede observar en la matriz, existe una correlación lineal débil entre la mayoría de las variables independientes y la variable objetivo *DepDelay*. Las excepciones relativas son *CRSDepTime* y *CRSArrTime*, que presentan los valores positivos más altos (0.12 y 0.11 respectivamente), aunque su poder predictivo lineal sigue siendo moderado[cite: 3]. Asimismo, se detectó una fuerte multicolinealidad (0.70) entre *CRSDepTime* y *CRSArrTime*; un comportamiento estadísticamente esperado dado que representan ventanas temporales lógicamente secuenciales de un mismo vuelo.

1.3.1. Conclusiones

A partir de este análisis, podemos determinar que:

Naturaleza de los datos: La baja correlación generalizada sugiere que la relación que dicta los retrasos no es estrictamente lineal. La baja correlación observada entre las variables predictoras y *DepDelay* sugiere que una parte importante de la variabilidad de los retrasos podría no ser explicada mediante relaciones lineales simples. Por ello, es razonable esperar que el coeficiente de determinación R^2 obtenido por una regresión lineal sea limitado.

Multicolinealidad temporal: La fuerte

relación entre las horas de salida y llegada implica que ambas variables aportan información redundante al modelo, lo cual podría generar cierta inestabilidad al momento de que el algoritmo asigne los pesos.

Disparidad de magnitudes: Las variables independientes operan en escalas drásticamente distintas, contrastando vuelos de miles de millas físicas contra el formato temporal de horas y minutos.

Por estas razones, y para resolver específicamente la diferencia de magnitudes, es indispensable proceder con la normalización de los datos. Este paso asegurará que todas las características numéricas convivan en un rango equitativo. Al nivelar la escala, evitamos que las variables con valores más grandes dominen el cálculo de la función de costo, permitiendo que el algoritmo del *Descenso de Gradiente* converja hacia los pesos óptimos de manera mucho más rápida y estable. En la siguiente sección, hablaré un poco más a detalle sobre los algoritmos que aplicaré sobre mi modelo y su función dentro del mismo.

1.4. Modelo de regresión lineal sin frameworks

1.4.1. Normalización de los datos

El algoritmo a utilizar para la normalización de los datos será *Min-Max*:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Gracias a este algoritmo, lograremos poner en una escala del 0 a 1 todos nuestros datos y, como ya se ha dicho anteriormente, durante la aplicación del *Descenso del Gradiente* converja más rápido.

Hay que tener cuidado a qué variables les aplicamos la normalización de los datos

pues no es necesarios hacerlo para todas las variables. Además, la normalización se debe de hacer con respecto a los datos de entrenamiento, pero aplicárselo a todos los conjuntos de datos (*test* y *validation*). Como anteriormente he aplicado un *One Hot Encoding* para las variables temporales, entonces, las úncias columnanas a las que se les aplicará la normalización serán a *CRSDepTime*, *CRSArrTime* y *Distance*.

1.4.2. Implementación del modelo

Una vez preparados y normalizados los datos, se implementó el modelo de regresión lineal a partir de las variables predictoras. Para una observación con n características, la predicción se expresa como:

$$\hat{y} = b + \sum_{j=1}^n w_j x_j \quad (1.1)$$

donde $\hat{y}$ representa la predicción del modelo, b el bias, w_j el peso asociado a la característica x_j , y n el número total de variables predictoras. A continuación se describen las funciones que componen esta implementación.

Función de hipótesis y cálculo del error

La función de hipótesis recibe las características de una observación junto con los pesos y el bias actuales, y devuelve una predicción:

$$h(x) = b + \sum_{j=1}^n w_j x_j \quad (1.2)$$

Sobre esta predicción se calcula el error de cada observación como la diferencia entre el valor predicho y el real:

$$e_i = \hat{y}_i - y_i \quad (1.3)$$

Tanto la hipótesis como el error son la base del resto de la implementación, ya que el

cálculo del costo y de los gradientes dependen directamente de ellos.

Función de costo

Para medir qué tan alejadas se encuentran las predicciones de los valores reales se utilizó el *Mean Squared Error* (MSE):

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 \quad (1.4)$$

donde m representa el número de observaciones utilizadas. Elevar los errores al cuadrado evita que los errores positivos y negativos se cancelen entre sí, y penaliza en mayor medida los errores de mayor magnitud. El objetivo del Descenso de Gradiente es encontrar los valores de w y b que minimicen esta función.

Gradientes de los parámetros

Para actualizar los parámetros del modelo es necesario conocer cómo cambia la función de costo respecto a cada uno de ellos. Las derivadas parciales del costo respecto a un peso w_j y respecto al bias son:

$$\frac{\partial J}{\partial w_j} = \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij} \quad (1.5)$$

$$\frac{\partial J}{\partial b} = \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) \quad (1.6)$$

Ambas expresiones se calculan recorriendo todas las observaciones del lote y acumulando el error correspondiente; la única diferencia es que, en el caso del bias, el error no se multiplica por ninguna característica, ya que la derivada de la hipótesis respecto a b es igual a 1. El resultado de cada gradiente indica la dirección y magnitud en la que debe modificarse el parámetro correspondiente para disminuir el costo.

Actualización de los parámetros

Con los gradientes calculados, los parámetros se actualizan mediante el algoritmo de

Descenso de Gradiente:

$$w_j^{\text{nuevo}} = w_j - \alpha \frac{\partial J}{\partial w_j} \quad (1.7)$$

$$b^{\text{nuevo}} = b - \alpha \frac{\partial J}{\partial b} \quad (1.8)$$

donde α representa la tasa de aprendizaje (*learning rate*). El signo negativo es fundamental, ya que el gradiente indica la dirección de mayor incremento del costo; por lo tanto, se avanza en la dirección contraria para acercarse a un mínimo de la función.

Entrenamiento por mini-batch

Debido a que el conjunto de entrenamiento contiene aproximadamente 9.8 millones de observaciones, calcular los gradientes utilizando todos los datos antes de cada actualización resultaba computacionalmente costoso. Por ello se utilizó *Mini-Batch Gradient Descent*, dividiendo el conjunto de entrenamiento en lotes de 2048 observaciones. Esto permite actualizar los parámetros varias veces por época utilizando solo una fracción de los datos en cada paso, logrando un balance entre el costo computacional de usar todos los datos a la vez y la inestabilidad de actualizar con una sola observación.

Durante cada época se recorren todos los lotes, repitiendo para cada uno el ciclo: predicción → error → costo → gradientes → actualización de w y b . Al finalizar la época se promedia el costo de entrenamiento entre todos los lotes.

Uso de los conjuntos de entrenamiento, validación y prueba

El conjunto de *train* es el único utilizado para actualizar los pesos y el bias en cada época. El conjunto de *validation*, en cambio, no participa en esta actualización: después de cada época se usan los parámetros obtenidos hasta ese momento para generar predicciones sobre él y calcular su costo, lo que permite observar qué tanto generaliza el modelo a datos no usados para ajustarlo.

El conjunto de *test* se mantiene completamente al margen durante el entrenamiento y solo se utiliza al final, con los parámetros ya definidos, para obtener una evaluación final e independiente del desempeño del modelo.

1.4.3. Resultados del modelo

Para el entrenamiento del modelo, utilicé los siguientes parámetros:

- *learning rate* = 0,01
- *epochs* = 10
- *batch size* = 2048

Graficando la evolución del *MSE* por cada época, resultó en lo siguiente:

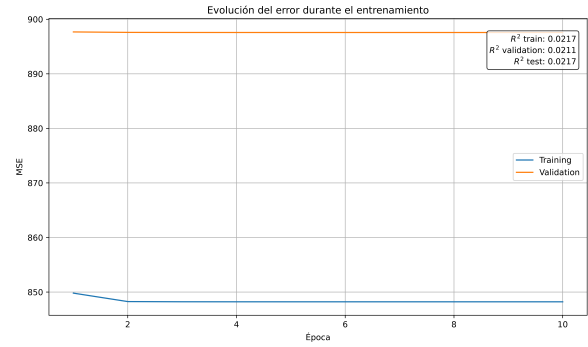


Figura 1.5: Evolución del *Mean Squared Error* por cada época.

El modelo alcanzó un R^2 de 0.0216 en el conjunto de *train*, valor consistente con la baja correlación lineal observada desde el *EDA* en la matriz de correlación (ver figura ??), donde la correlación máxima llegaba a 0.12. Además, podemos observar claramente que tenemos un problema de *underfitting* en nuestro modelo. Esto confirma que la limitación no proviene de un error en el pipeline de datos ni en la implementación del algoritmo, sino de la naturaleza no lineal del fenómeno de retrasos en los vuelos, que una sola neurona no puede capturar más allá de esta cota.

Aunque no podemos graficar una comparación entre los valores X vs Y junto con los

X vs $\hat{Y}$, porque la dimensión de las variables predictorias es mayor a 3, un gráfico interesante podría ser comparando en dónde es, simplemente, comprar Y vs $\hat{Y}$ y Y con ella misma. De esta forma, podemos visualizar de forma sencilla si es que realmente los datos que se está prediciendo son realmente los que deberían de ser. El resultado de hacer esto es el siguiente (ver figura ??):

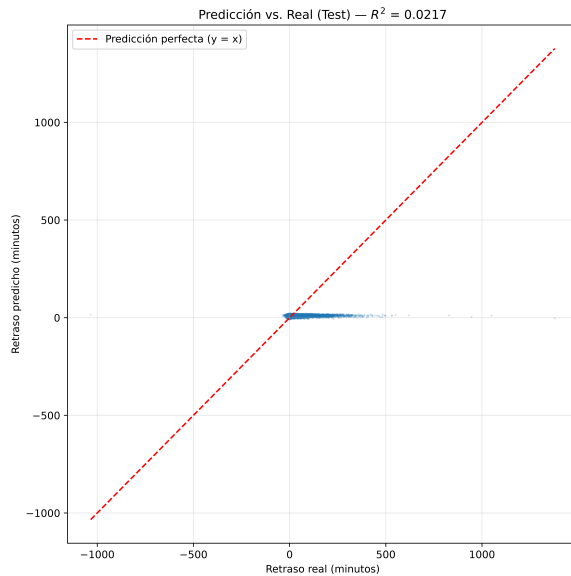


Figura 1.6: Valor predicho contra el actual.

Cabe mencionar que, para esta primera implementación, el conjunto de *validation* únicamente se utilizó con fines de monitoreo. Esto con la intención de verificar que el modelo generaliza de forma consistente y descartar *overfitting*, más que para una selección de hiperparámetro, la cuál se abordará en el siguiente capítulo.

Capítulo 2

Modelo con *frameworks*

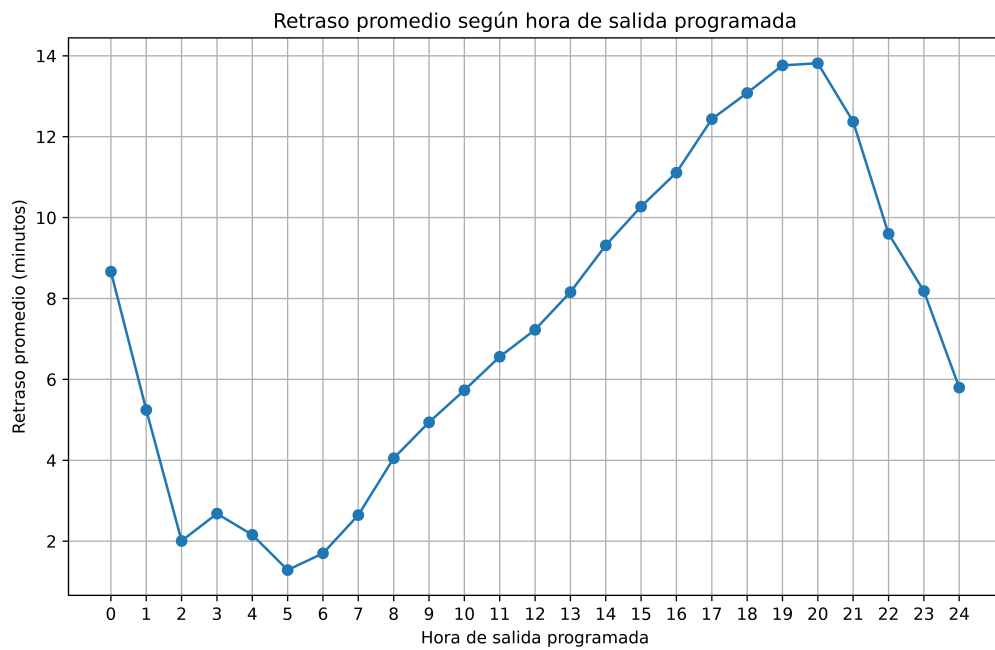
TBD

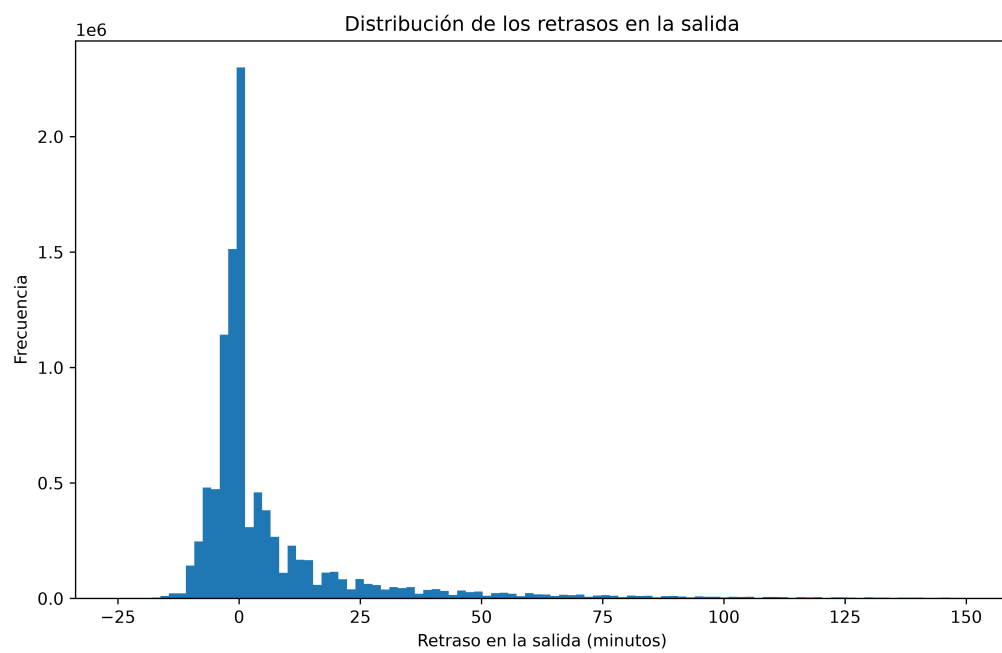
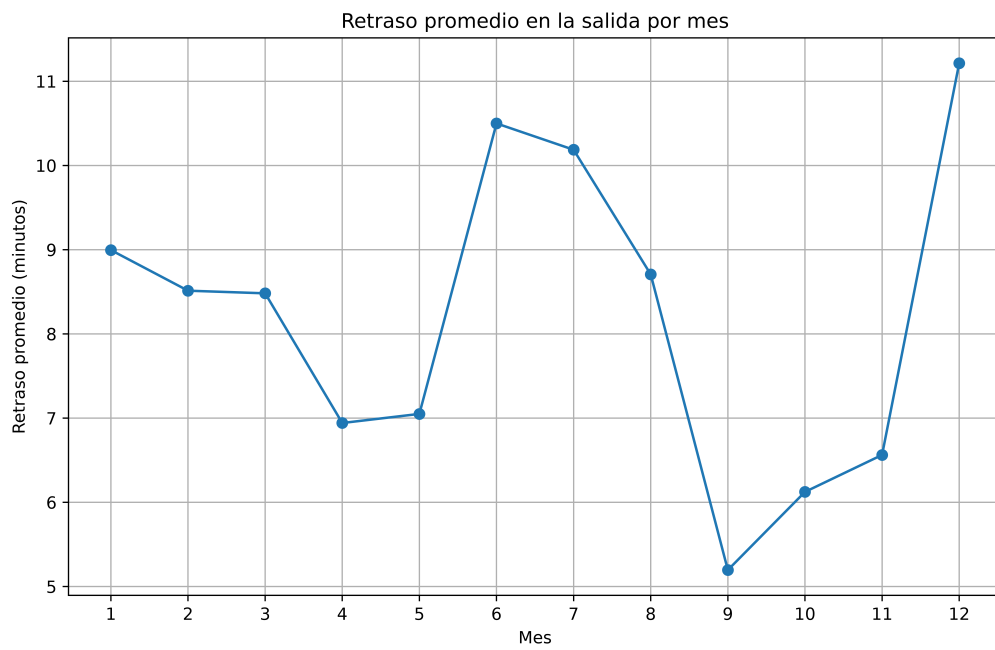
Apéndice A

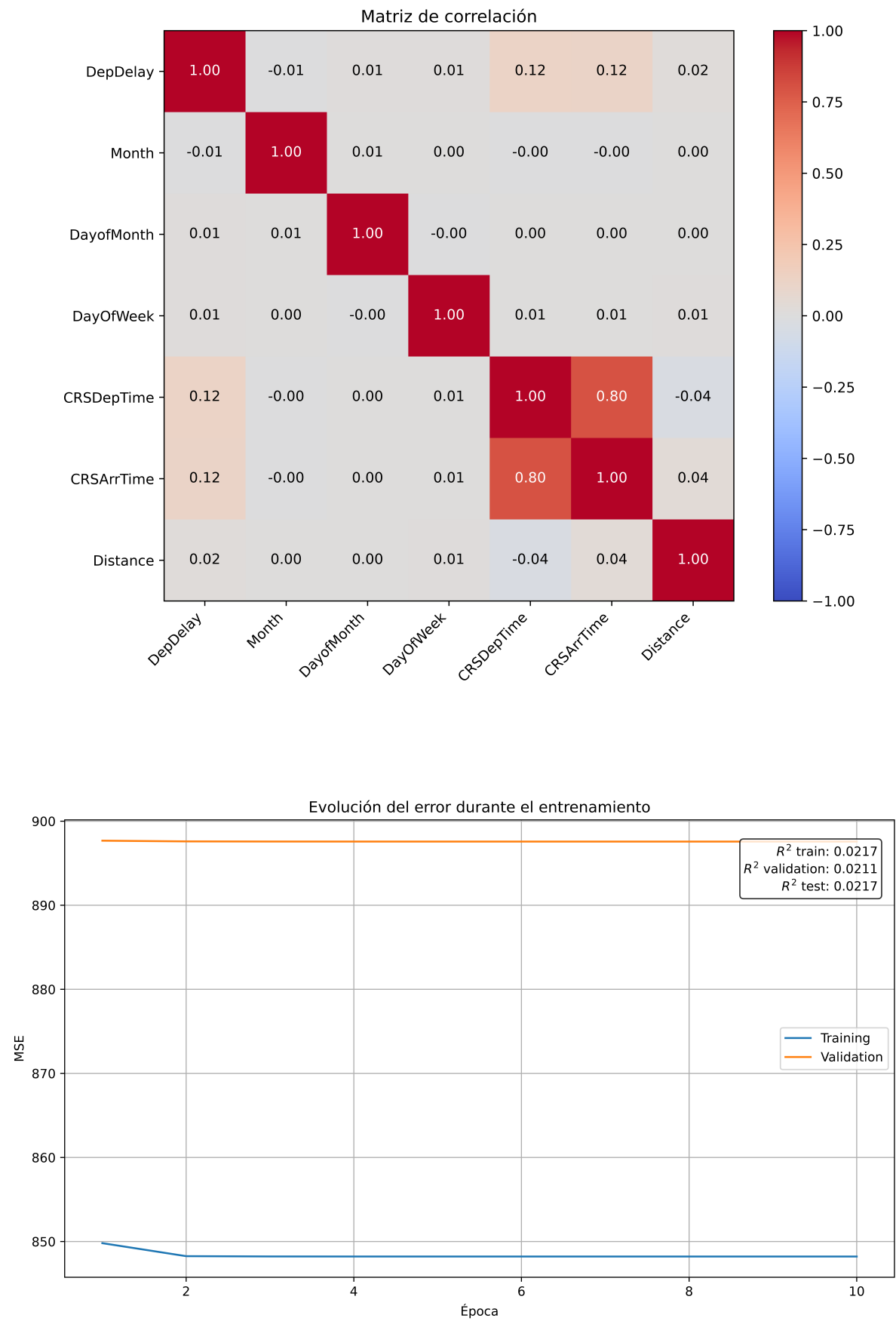
Figuras ampliadas

A continuación, se vuelven a dejar todas las figuras que contienen gráficos en un mayor tamaño poder visualizarlos mejor.









Apéndice B

Códigos

El siguiente código sirve únicamente para la generación de los gráficos presentados, aunque no todos fueron utilizados y/o fueron remplazados.

```
1 # Archivo auxiliar para generar imagenes.
2
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import pandas as pd
6 import seaborn as sns
7
8 DPI = 900 # Resolución final de los gráficos
9
10 def correlation_matrix(df: pd.DataFrame):
11     columns = [
12         "DepDelay",
13         "Month",
14         "DayofMonth",
15         "DayOfWeek",
16         "CRSDepTime",
17         "CRSArrTime",
18         "Distance"
19     ]
20     corr = df[columns].corr()
21     plt.figure(figsize=(9, 7))
22     plt.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
23     plt.colorbar()
24     plt.xticks(range(len(columns)), columns, rotation=45, ha="right")
25     plt.yticks(range(len(columns)), columns)
26     for i in range(len(columns)):
27         for j in range(len(columns)):
28             plt.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center",
29                     va="center", color="white" if abs(corr.iloc[i, j])
30                     > 0.6 else "black")
31     plt.title("Matriz de correlación")
32     plt.tight_layout()
33     plt.savefig("./graficas/correlation_matrix.png", dpi=DPI)
34     plt.close()
```

```

33
34
35 def delays_hist(df:pd.DataFrame):
36     plt.figure(figsize=(10, 6))
37     bins = int(np.sqrt(len(df)))
38     plt.hist(df["DepDelay"], bins=100, range=(-25, 150))
39     plt.xlabel("Retraso en la salida (minutos)")
40     plt.ylabel("Frecuencia")
41     plt.title("Distribución de los retrasos en la salida")
42     plt.savefig("./graficas/depdelay_histogram.png", dpi=DPI)
43     plt.close()
44
45
46 def mean_delay_by_month(df: pd.DataFrame):
47     monthly_mean = df.groupby("Month")["DepDelay"].mean()
48     plt.figure(figsize=(10, 6))
49     plt.plot(monthly_mean.index, monthly_mean.values, marker="o")
50     plt.xlabel("Mes")
51     plt.ylabel("Retraso promedio (minutos)")
52     plt.title("Retraso promedio en la salida por mes")
53     plt.xticks(range(1, 13))
54     plt.grid()
55     plt.savefig("./graficas/mean_depdelay_by_month.png", dpi=DPI)
56     plt.close()
57
58
59 def mean_delay_by_week(df: pd.DataFrame):
60     weekly_mean = df.groupby("DayOfWeek")["DepDelay"].mean()
61     plt.figure(figsize=(10, 6))
62     plt.plot(weekly_mean.index, weekly_mean.values, marker="o")
63     plt.xlabel("Día de la semana")
64     plt.ylabel("Retraso promedio (minutos)")
65     plt.title("Retraso promedio en la salida por día de la semana")
66     plt.xticks(range(1, 8))
67     plt.grid()
68     plt.savefig("./graficas/mean_depdelay_by_weekly.png", dpi=DPI)
69     plt.close()
70
71
72 def mean_delay_by_hour(df: pd.DataFrame):
73     temp = df.copy()
74     temp["Hour"] = (temp["CRSDepTime"] // 60).astype(int)
75     hourly_mean = temp.groupby("Hour")["DepDelay"].mean()
76     plt.figure(figsize=(10, 6))
77     plt.plot(hourly_mean.index, hourly_mean.values, marker="o")
78     plt.xlabel("Hora de salida programada")
79     plt.ylabel("Retraso promedio (minutos)")
80     plt.title("Retraso promedio según hora de salida programada")
81     plt.xticks(range(25))
82     plt.grid()

```

```

83     plt.savefig("./graficas/mean_depdelay_by_hour.png", dpi=DPI)
84     plt.close()
85
86 def pair_plot(df:pd.DataFrame):
87     print(df)
88     df = df.iloc[:, 0:7]
89     sns.pairplot(df, hue="DayOfWeek").savefig("./graficas/
90         pair_plot.png", dpi=DPI)
91
92 def predicted_vs_actual(y_real: list, y_pred: list, r2: float,
93     dataset_name="Test"):
94     plt.figure(figsize=(8, 8))
95     plt.scatter(y_real, y_pred, s=1, alpha=0.15)
96     plt.legend()
97     lims = [min(min(y_real), min(y_pred)), max(max(y_real), max(
98         y_pred))]
99     plt.plot(lims, lims, 'r--', label="Predicción perfecta (y = x)
100         ")
101     plt.xlabel("Retraso real (minutos)")
102     plt.ylabel("Retraso predicho (minutos)")
103     plt.title(f"Predicción vs. Real ({dataset_name})      $R^2$ = {
104         r2:.4f}")
105     plt.legend()
106     plt.grid(alpha=0.3)
107     plt.tight_layout()
108     plt.savefig(f"./graficas/predicted_vs_actual_{dataset_name}.
109         lower().png", dpi=DPI)
110     plt.close()

```

Proceso de ETL.

```

1  import os
2  import pandas as pd
3  import numpy as np
4  from scipy.io import arff # Para leer el tipo de archivo en el que
5     viene el dataset
6
7  def hhmm_to_minutes(time):
8     time = int(time)
9     hours = time // 100
10    minutes = time % 100
11    return hours * 60 + minutes
12
13 def extract_data():
14     print("Cargando los datos...")
15     df = None
16     if "airlines_10M.parquet" not in os.listdir():
17         arff_file = arff.loadarff('./
18             airlines_train_regression_10000000.arff')
19         df = pd.DataFrame(arff_file[0])
20         print("\nEstado final del dataset:")
21         print(df.info())

```

```

20     return df, False
21 else:
22     df = pd.read_parquet("airlines_10M.parquet", engine="
        pyarrow")
23     print("\nDatos generales:")
24     print(df.info())
25     print("Saltando transformación de los datos. Ya se han
        sido limpiados.")
26     return df, True
27
28 def transform_data(df:pd.DataFrame):
29     print("\nExcluyendo columnas UniqueCarrier, Origin y Dest")
30
31     df = df.drop(columns=["UniqueCarrier", "Origin", "Dest"])
32
33     print("\nConvirtiendo CRSDepTime y CRSArrTime a minutos desde
        la medianoche...")
34
35     df["CRSDepTime"] = df["CRSDepTime"].apply(hhmm_to_minutes)
36     df["CRSArrTime"] = df["CRSArrTime"].apply(hhmm_to_minutes)
37
38     print("Conversión realizada.")
39
40     print("Datos generales:")
41
42     print(df.info())
43     print(df.describe())
44     print("Cantidad de valores nulos:")
45     print(df.isna().sum())
46     print("Nota: No existen valores nulos :D")
47
48     print("Aplicando One Hot Encoding para columnas DayOfWeek,
        DayofMonth y Month...")
49     dummies = pd.get_dummies(df[["DayOfWeek", "DayofMonth", "Month
        "]], columns=["DayOfWeek", "DayofMonth", "Month"], dtype=int)
50     df = pd.concat([df, dummies], axis=1)
51     print("One Hot Encoding aplicado.")
52
53
54     print("Guardando datos en formato parquet para tener una carga
        más rápida en futuras pruebas...")
55     df.to_parquet("airlines_10M.parquet", engine="pyarrow")
56     print("Datos en parquet guardados.")
57
58     return df
59
60 def split_data(df:pd.DataFrame):
61     df = df.sample(frac=1).reset_index(drop=True)
62     Yvalues = df["DepDelay"]
63     Xvalues = df.drop(columns=["DepDelay", "DayOfWeek", "
        DayofMonth", "Month"])

```

```

64     df_len = len(df)
65     # Al ser un gran tamaño de datos, dividiré los datos 98/1/1
66     Xtrain = Xvalues[0:(df_len*98)//100]
67     Xvalidation = Xvalues[(df_len*98)//100:(df_len*99)//100]
68     Xtest = Xvalues[(df_len*99)//100:]
69     ytrain = Yvalues[0:(df_len*98)//100]
70     yvalidation = Yvalues[(df_len*98)//100:(df_len*99)//100]
71     ytest = Yvalues[(df_len*99)//100:]
72     # Regreso también el dataset completo para realizar gráficas
       en caso de ser necesario
73     return df, Xtrain, Xvalidation, Xtest, ytrain, yvalidation,
       ytest
74
75
76 def normalize_data(X_train: pd.DataFrame, X_val: pd.DataFrame,
77                   X_test: pd.DataFrame):
78     columns_to_normalize = ["CRSDepTime", "CRSArrTime", "Distance"
79                             ]
80     X_train_norm = X_train.copy()
81     X_val_norm = X_val.copy()
82     X_test_norm = X_test.copy()
83
84     # Min y max calculados con train
85     min_vals = X_train[columns_to_normalize].min()
86     max_vals = X_train[columns_to_normalize].max()
87
88     # Normalización
89     X_train_norm[columns_to_normalize] = (X_train[
90         columns_to_normalize] - min_vals) / (max_vals - min_vals)
91     X_val_norm[columns_to_normalize] = (X_val[
92         columns_to_normalize] - min_vals) / (max_vals - min_vals)
93     X_test_norm[columns_to_normalize] = (X_test[
94         columns_to_normalize] - min_vals) / (max_vals - min_vals)
95
96     return X_train_norm, X_val_norm, X_test_norm

```

Implementación del modelo de regresión lineal.

```

1  import math
2  from tqdm import tqdm # Para crear una barra de progreso al
   momento de entrenar el modelo
3
4  def hypo(params: list, weights: list, bias: float) -> float:
5      result = 0
6      for i in range(len(params)):
7          result += params[i] * weights[i]
8      return result + bias
9
10 def error(y_pred:float,y_real:float)->float:
11     return y_pred - y_real
12
13 def cost(y_hat:list, y:list)->float:

```

```
14     result = 0
15     for i in range(len(y)):
16         result += (error(y_hat[i], y[i]))**2
17     result = 1/len(y) * result
18     return result
19
20 def r_squared(y_pred: list, y_real: list) -> float:
21     n = len(y_real)
22     mean_y = sum(y_real) / n
23     ss_res = 0.0
24     ss_tot = 0.0
25     for i in range(n):
26         ss_res += (y_real[i] - y_pred[i])**2
27         ss_tot += (y_real[i] - mean_y)**2
28     return 1 - (ss_res / ss_tot)
29
30 def gradient(weights: list, bias: float, params: list, y: list):
31     gradients = [0] * len(weights)
32     bias_gradient = 0
33     m = len(params)
34
35     for i in range(m):
36         predict = hypo(params[i], weights, bias)
37         predict_error = error(predict, y[i])
38
39         for j in range(len(weights)):
40             gradients[j] += predict_error * params[i][j]
41         bias_gradient += predict_error
42
43     for j in range(len(weights)):
44         gradients[j] = (2/m) * gradients[j]
45     bias_gradient = (2/m) * bias_gradient
46
47     return gradients, bias_gradient
48
49
50 def update(weights: list, bias: float, gradients: list, bias_gradient
51 : float, lr: float):
52     new_weights = []
53     for i in range(len(weights)):
54         new_weight = weights[i] - (lr * gradients[i])
55         new_weights.append(new_weight)
56     new_bias = bias - (lr * bias_gradient)
57     return new_weights, new_bias
58
59 def predict(params: list, weights: list, bias: float):
60     predictions = []
61     for i in range(len(params)):
62         prediction = hypo(params[i], weights, bias)
63         predictions.append(prediction)
```

```

64     return predictions
65
66
67 def train_step(weights:list, bias:float, params:list, y:list, lr:
float):
68     predictions = predict(params, weights, bias)
69     current_cost = cost(predictions, y)
70     gradients, bias_gradient = gradient(weights, bias, params, y)
71     new_weights, new_bias = update(weights, bias, gradients,
        bias_gradient, lr)
72     return new_weights, new_bias, current_cost
73
74 def train(params:list, y:list, validation_params:list,
validation_y:list, lr:float, epochs:int, batch_size:int):
75     weights = [0 for _ in range(len(params[0]))]
76     bias = 0
77     train_costs = []
78     validation_costs = []
79
80     m = len(params)
81     num_batches = math.ceil(m / batch_size)
82
83     total_steps = epochs*num_batches
84     pbar = tqdm(total=total_steps, desc="Entrenando modelo")
85
86     for epoch in range(epochs):
87         epoch_train_cost = 0
88         # Iteramos sobre los datos en pequeños lotes
89         for i in range(num_batches):
90             # Calculamos los índices de inicio y fin
91             start_idx = i * batch_size
92             end_idx = min(start_idx + batch_size, m)
93             # Extraemos el lote actual
94             batch_params = params[start_idx:end_idx]
95             batch_y = y[start_idx:end_idx]
96             # Entrenamos sólo con este lote.
97             weights, bias, batch_cost = train_step(weights, bias,
                batch_params, batch_y, lr)
98             # Sumamos el costo del lote para sacar el promedio
                después
99             epoch_train_cost += batch_cost
100             # Actualizar barra de progreso
101             pbar.update(1)
102             pbar.set_postfix({
103                 "ep": f"{epoch+1}/{epochs}",
104                 "batch": f"{i+1}/{num_batches}",
105             })
106
107             # Costo promedio de entrenamiento de toda la época
108             epoch_train_cost /= num_batches
109             train_costs.append(epoch_train_cost)

```

```
110
111     # Evaluamos en validación
112     validation_predictions = predict(validation_params,
113                                     weights, bias)
114     validation_cost = cost(validation_predictions,
115                             validation_y)
116     validation_costs.append(validation_cost)
117
118     return weights, bias, train_costs, validation_costs
```

Código principal.

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 from plotUtil import *
5 from etlProcess import *
6 from regressionModel import train, predict, cost, r_squared
7
8 def initial_graphs(df:pd.DataFrame):
9     # pair_plot(df) # Por el tiempo de computo, preferible sólo
10     # realizarlo una única vez.
11     mean_delay_by_hour(df)
12     mean_delay_by_month(df)
13     mean_delay_by_week(df)
14     correlation_matrix(df)
15
16 def etl_process():
17     # ETL
18     ## Extracting
19     df, _skip = extract_data()
20     if not _skip:
21         ## Transform
22         df = transform_data(df)
23     # Loading
24     # Como tal, no se subirán los datos a algun repositorio u otro
25     # lugar, sin embargo
26     # sirve realizar en esta sección la separación de los datos.
27     df, Xtrain, Xvalidation, Xtest, ytrain, yvalidation, ytest =
28     split_data(df)
29     print("Datos separados en train (98%), validation (1%) y test
30     (1%)")
31     print("Normalizando las columnas CRSDepTime, CRSArrTime y
32     Distance...")
33     Xtrain_norm, Xvalidation_norm, Xtest_norm = normalize_data(
34     Xtrain, Xvalidation, Xtest)
35
36     return df, Xtrain_norm, Xvalidation_norm, Xtest_norm, ytrain,
37     yvalidation, ytest
38
39 def main():
```

```
34 # ETL
35 df, Xtrain_norm, Xvalidation_norm, Xtest_norm, ytrain,
    yvalidation, ytest = etl_process()
36
37 _plot_graps = False # cambiar a True en caso de querer
    graficar
38 if _plot_graps:
39     print("Graficando parte análisis...")
40     initial_graphs(df)
41     print("Finalizando graficas iniciales.")
42
43 # Datos a utilizar.
44 # Nota: Esto sólo es para realizar pruebas rápidas por el tama
    ño del dataset.
45 # Al final, data_percentage debe de valer 1.0 = 100% de los
    datos.
46 data_percentage = 1.0
47
48 train_samples = int(len(Xtrain_norm) * data_percentage)
49 validation_samples = int(len(Xvalidation_norm) *
    data_percentage)
50 test_samples = int(len(Xtest_norm) * data_percentage)
51
52 print("\nPreparando datos para el modelo...")
53
54 params = Xtrain_norm.iloc[:train_samples].values.tolist()
55 y = ytrain.iloc[:train_samples].values.tolist()
56
57 validation_params = Xvalidation_norm.iloc[:validation_samples
    ].values.tolist()
58 validation_y = yvalidation.iloc[:validation_samples].values.
    tolist()
59
60 test_params = Xtest_norm.iloc[:test_samples].values.tolist()
61 test_y = ytest.iloc[:test_samples].values.tolist()
62
63 print("Porcentaje de datos utilizado:", data_percentage * 100,
    "%")
64 print("Número de observaciones de entrenamiento:", len(params)
    )
65 print("Número de variables:", len(params[0]))
66 print("Número de observaciones de validation:", len(
    validation_params))
67 print("Número de observaciones de test:", len(test_params))
68
69 # Parámetros del modelo
70 lr = 0.01
71 epochs = 10
72 batch_size = 2048
73
74 # Entrenamiento
```

```

75     weights, bias, train_costs, validation_costs = train(params, y
76         , validation_params, validation_y, lr, epochs, batch_size)
77
78     # Resultados
79     print("\nParámetros finales:")
80     print("Weights:", weights)
81     print("Bias:", bias)
82
83     # Costos de train y validation
84
85     print("\nCosto de entrenamiento:")
86     print("Inicial:", train_costs[0])
87     print("Final:", train_costs[-1])
88
89     print("\nCosto de validation:")
90     print("Inicial:", validation_costs[0])
91     print("Final:", validation_costs[-1])
92
93     # Evaluación final en test
94     test_predictions = predict(test_params, weights, bias)
95     test_cost = cost(test_predictions, test_y)
96
97     # R^2 para cada conjunto
98     train_predictions_final = predict(params, weights, bias)
99     validation_predictions_final = predict(validation_params,
100         weights, bias)
101
102     train_r2 = r_squared(train_predictions_final, y)
103     validation_r2 = r_squared(validation_predictions_final,
104         validation_y)
105     test_r2 = r_squared(test_predictions, test_y)
106
107     print("\nR^2:")
108     print("Train:", train_r2)
109     print("Validation:", validation_r2)
110     print("Test:", test_r2)
111
112     print(f"{'Real':>10} {'Predicho':>10} {'Error':>10}")
113     for i in range(15):
114         real = test_y[i]
115         pred = test_predictions[i]
116         print(f"{real:>10.2f} {pred:>10.2f} {pred - real:>10.2f}")
117
118     # Gráfica de predicción vs. real
119     predicted_vs_actual(test_y, test_predictions, test_r2,
120         dataset_name="Test")
121
122     # Gráfica de training y validation
123     plt.figure(figsize=(10, 6))
124     plt.plot(range(1, len(train_costs) + 1), train_costs, label="

```

```
    Training")
122 plt.plot(range(1, len(validation_costs) + 1), validation_costs
    , label="Validation")
123 plt.xlabel("Época")
124 plt.ylabel("MSE")
125 plt.title("Evolución del error durante el entrenamiento")
126 r2_text = (f"$R^2$ train: {train_r2:.4f}\n" f"$R^2$ validation
    : {validation_r2:.4f}\n" f"$R^2$ test: {test_r2:.4f}")
127 plt.gca().text(
128     0.98, 0.95, r2_text,
129     transform=plt.gca().transAxes,
130     ha="right", va="top",
131     fontsize=10,
132     bbox=dict(boxstyle="round", facecolor="white", alpha=0.8)
133 )
134 plt.legend()
135 plt.grid()
136 plt.tight_layout()
137 plt.savefig("./graficas/cost_evolution.png", dpi=DPI)
138
139
140
141
142 if __name__ == '__main__':
143     main()
```

Bibliografía

- [1] Matthew Kosinski Annie Badman. *¿Qué es Apache Parquet? / IBM* — *ibm.com*. <https://www.ibm.com/mx-es/think/topics/parquet>. [Accessed 26-08-2026].
- [2] scgupta. *Training, Validation, and Test Split for ML Models* — *ml4devs.com*. <https://www.ml4devs.com/what-is/train-validation-test-data-split/>. [Accessed 26-08-2026].